TP - Préparation à la soutenance

malek.bengougam@gmail.com

Partie 1 – Matrices monde

La première transformation que l'on rencontre usuellement consiste à placer un objet dans la scène.

La « World Matrix », appelée aussi « Local-To-World » ou « Object-To-World », contient la concaténation des différentes transformations linéaires (scale, rotation) et affine (translation).

Jusqu'à présent nous avons séparés ces différentes transformation en matrices distinctes, mais pour des raisons de performance et de simplification du code il est habituel de réduire ces transformations à une seule matrice envoyée au shader.

Cela permet ainsi de réduire le nombre de matrices transmises au GPU (d'autant que la mémoire pour les constantes uniform-s est relativement limitée) mais également d'augmenter les performances en réduisant le nombre de multiplications matricielles qui s'éffectuent à chaque invocation du Vertex Shader.

Exercice 1.1 : implémenter en C ou C++ une fonction multipliant deux matrices et qui renvoit le résultat.

A titre de rappels, les matrices sont des matrices homogènes 4D stockées en mémoire en colonne d'abord (première colonne, puis seconde colonne, etc...).

L'ordre des transformations se lit de la droite vers la gauche. En infographie, pour la "World Matrix" on concatène les matrices suivantes (dans cet ordre) :

Eq 1.1: World Matrix = Translation Matrix * Rotation Matrix * Scale Matrix

Exercice 1.2 : remplacer vos matrices de transformation locale par une unique transformation locale-vers-monde.

Rappel important*: TOUT point ou vecteur devant être transformé du repère local de l'objet vers le repère de la scène DOIT être transformé par la World Matrix*

Ceci vaut bien évidemment pour les positions des Vertex, mais également pour les normales.
Comme vue brièvement lors du TP précédent, les normales doivent être transformées par la transposée de l'inverse de la World Matrix c'est-à-dire

Eq 1.2: Normal Matrix = (World Matrix^{-1})^{T} , en GLSL: NormalMatrix = **transpose**(**inverse**(WorldMatrix));

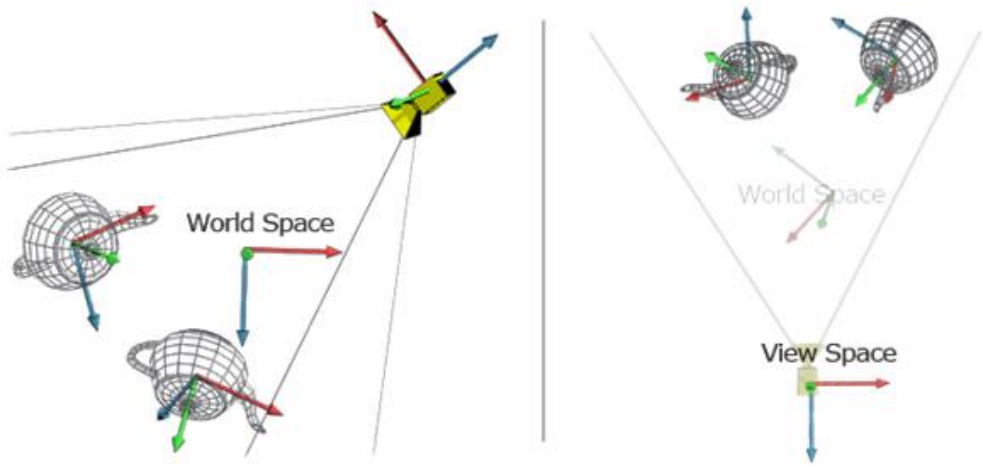
Il faut également noter que la translation n'a pas de sens pour une normale, c'est pour cela qu'une normale se représente en coordonnée homogène (4D) avec une composante **w** à **zéro**. Mais le plus simple est ici de considérer NormalMatrix comme une **mat3**, donc de convertir WorldMatrix en **mat3**.

Partie 2 – Matrice vue

La caméra est également un objet de la scène, et dispose donc d'une matrice World Matrix.

Pour simuler le fait que les objets sont vus depuis le point de vue de la caméra il faut également appliquer une transformation supplémentaire à l'ensemble des objets de la scène.

Cette transformation doit transformer du repère de la scène (world space) vers le repère de la caméra (aussi appelé "view space", "eye space" ou "camera space").



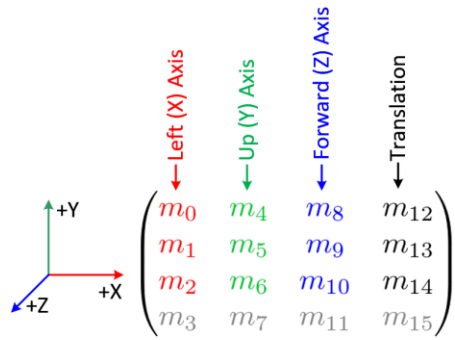
En fait, le repère de la caméra n'est rien d'autre que le repère local de la caméra. A savoir que dans ce repère, la caméra se trouve toujours située à l'origine.
(Pour une illustration voir cf. https://webglfundamentals.org/webgl/lessons/webgl-3d-camera.html)

Pour simuler une caméra il suffit de calculer la transformation (inverse) Monde-Vers-Objet, de la World Matrix de la caméra.
Cela nécessite de faire appel (et programmer) une fonction inverse en C++ mais on peut faire mieux ici.

Une caméra ne subit pas de scale pas plus que de déformations de ses axes. C'est donc une transformation orthogonale. Ici, WorldMatrix_{cam} la transformation Locale-vers-Monde de la caméra :

$\text{WorldMatrix}_{cam} = \text{TranslationMatrix}_{cam} * \text{RotationMatrix}_{cam}$

$$\begin{bmatrix} right_x & up_x & forward_x & position_x \\ right_y & up_y & forward_y & position_y \\ right_z & up_z & forward_z & position_z \\ 0 & 0 & 0 & 1 \end{bmatrix}$$



Visuellement on a une matrice homogène qui mélange changement de base dans la sous-matrice 3x3 - chaque colonne représente l'orientation respectives des axes de la base- et une translation.

Comme on a seulement des rotations et des translations, on peut très bien se contenter d'inverser ces matrices pour calculer $WorldMatrix_{cam}^{-1}$ l'inverse de $WorldMatrix_{cam}$:

$$WorldMatrix_{cam}^{-1} = (TranslationMatrix_{cam} * RotationMatrix_{cam})^{-1}$$
$$= RotationMatrix_{cam}^{-1} * TranslationMatrix_{cam}^{-1}$$

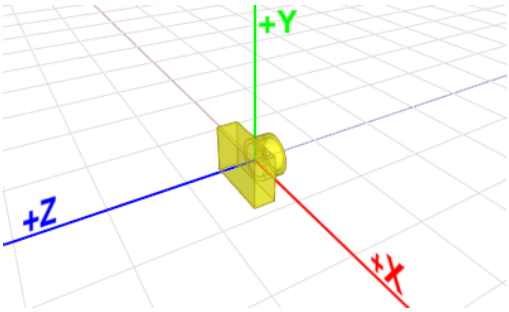
Une autre propriétés utiles, l'inverse d'une transformation linéaire orthogonale est une transposée. Et l'inverse d'une translation est simplement la translation opposée $T^{-1} = -T$. On optimise alors

$$ViewMatrix = WorldMatrix_{cam}^{-1} = RotationMatrix_{cam}^{T} * TranslationMatrix_{cam}^{-1}$$

$$\text{View Matrix: } \boldsymbol{M_{view}} = \boldsymbol{R}\,\boldsymbol{T} = \begin{bmatrix} x_x^c & x_y^c & x_z^c & 0 \\ y_x^c & y_y^c & y_z^c & 0 \\ z_x^c & z_y^c & z_z^c & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & -e_x \\ 0 & 1 & 0 & -e_y \\ 0 & 0 & 1 & -e_z \\ 0 & 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} x_x^c & x_y^c & x_z^c & -e_x \cdot x_c \\ y_x^c & y_y^c & y_z^c & -e_y \cdot y_c \\ z_x^c & z_y^c & z_z^c & -e_z \cdot z_c \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Important: remarquez que les éléments de la dernière colonne sont issues du produit scalaire entre le vecteur –e (-translation) et chacune des colonnes de la matrice (3x3) de rotation de la caméra R qui forment le repère de la caméra.

$$\begin{pmatrix} R^T & -R^T t \\ 0^T & 1 \end{pmatrix}$$



Fonction View Matrix "LookAt"

Plutôt que de raisonner en termes de compositions de matrices il est souvent plus facile d'utiliser des paramètres comme la position de la caméra et la cible (position) de la caméra.
On ajoute généralement un troisième paramètre référentiel "up" qui va nous permettre de former une base orthonormale.

Note: dans la plupart des cas la valeur du vecteur « up » correspond au « up » du monde soit [0, 1, 0]

ATTENTION *! Ne confondez pas notre fonction LookAt() OpenGL et d'autres fonctions telles Transform.LookAt() ou Matrix4x4.LookAt() de Unity.*
Elles prennent les mêmes paramètres mais cette dernière fonction génère une matrice Locale-vers-Monde qui permet de positionner un objet (par exemple une caméra) par rapport à un autre (la cible de la caméra) et non une matrice Vue qui est en fait l'inverse de Locale-vers-Monde.

Exercice 2.1: codez la fonction LookAt(vec3 position, vec3 target, vec3 up) et passez la matrice au Vertex Shader comme View Matrix.

Pour générer la “View Matrix” il nous faut les éléments suivants:

1. Vecteur “**forward**” de la nouvelle orientation. C’est le vecteur **-**(target-position) **normalisé**.

La base est exprimée en repère main-droite, le vecteur “forward” pointe ici HORS de l’écran. C’est pourquoi il y’a un signe moins (-).

2. Vecteur “**right**”. Il est formé par le **produit vectoriel** entre le vecteur “forward” et “up”.

Le produit vectoriel suit toujours la règle de la main-droite, à savoir, intuitivement, alignez le premier vecteur sur l’index de votre main droite, le second vecteur sur votre majeur, la direction du résultat est indiquée par votre pouce.

Ici, il faut donc que le premier vecteur soit “up” et le second “forward”.

3. Vecteur “**up**” corrigé. Il est formé par le **produit vectoriel** entre le vecteur “right” et “forward”.

Pareil ici, mais par commodité on va aligner le premier vecteur sur le pouce de votre main droite, le second vecteur sur votre index, la direction du résultat est indiquée par votre majeur.

Ici, faut donc que le premier vecteur soit “forward” et le second “right”.

4. Calculez **trois produits scalaires:** un entre la “position” et le vecteur “right”, un entre la “position” et le vecteur “up”, et un entre la “position” et le vecteur “forward”.

Cette opération correspond à une projection de la position (exprimée en repère monde) dans le repère local de la caméra afin de former une translation inverse. N’oubliez pas la négation des composants !

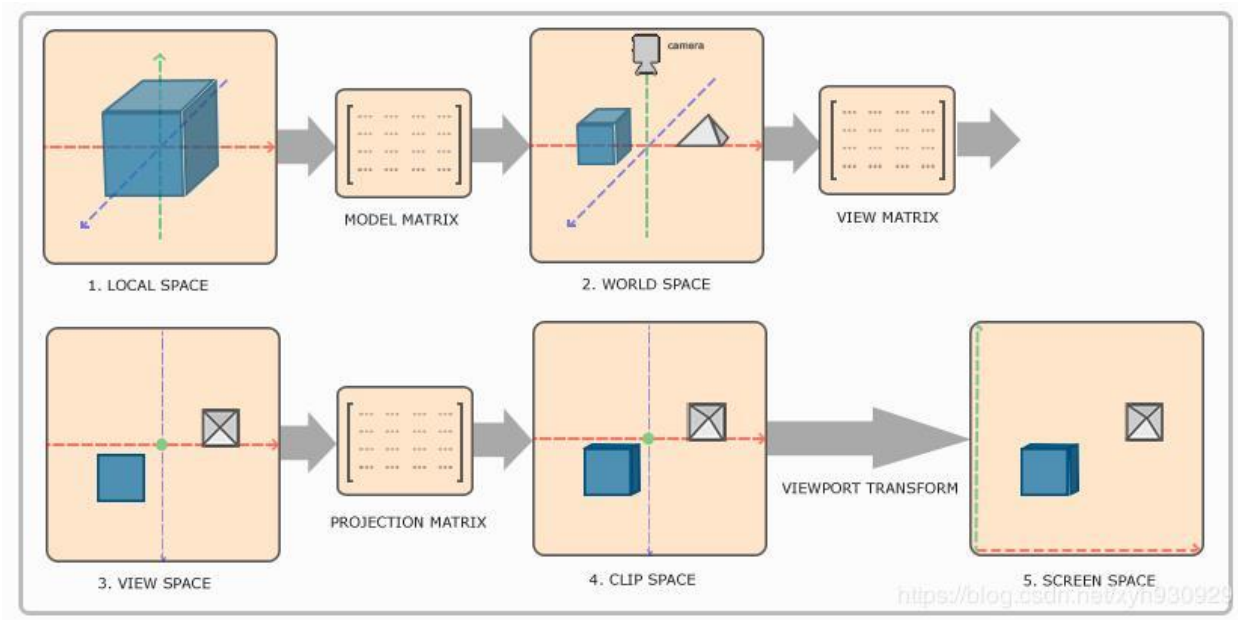
5. Assemblez le tout dans une matrice.

Les 3 premières colonnes contiennent la transposée des 3 vecteurs. On peut voir que la première colonne contient toutes les composantes ‘x’ des vecteurs, la seconde colonne toutes les composantes ‘y' et la troisième colonne toutes les composantes ‘z’.

La quatrième colonne contient les résultats de la 4ième étape.

$$V = \begin{bmatrix} X_c.x & X_c.y & X_c.z & -dot(X_c, P_c) \\ Y_c.x & Y_c.y & Y_c.z & -dot(Y_c, P_c) \\ Z_c.x & Z_c.y & Z_c.z & -dot(Z_c, P_c) \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

On a maintenant réduit l'ensemble des transformations 3D -> NDC aux trois transformations "Locale-vers-Monde" représentée par la World Matrix, "Monde-vers-Camera" représentée par la View Matrix ainsi que "Vers-NDC" représentée par la Projection Matrix.



Références :

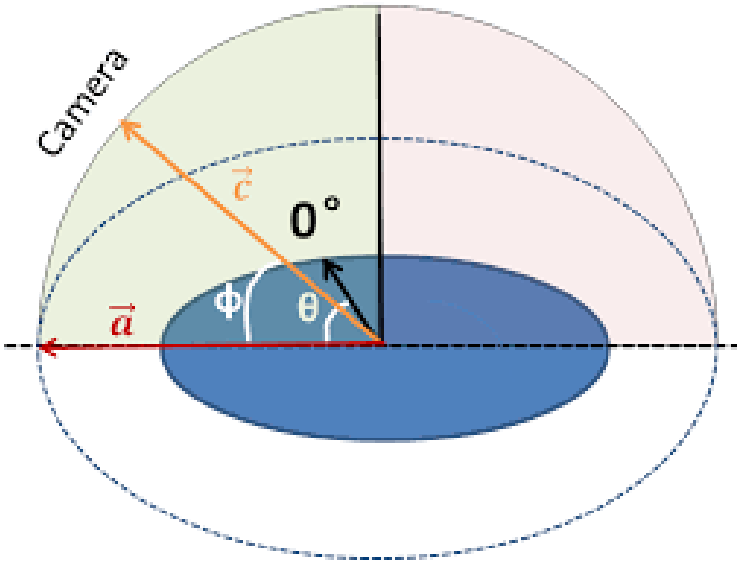
http://www.codinglabs.net/article_world_view_projection_matrix.aspx
http://www.opengl-tutorial.org/fr/beginners-tutorials/tutorial-3-matrices/

TP à rendre – Caméra orbitale et Illumination ambiante

1. Caméra orbitale

Implémentez une caméra orbitale type “arcball” avec GLFW. Les “inputs” sont les suivants:

- Axe horizontal de la souris, rotation autour du “up” (azimut, aussi appelée assiette)
- Axe vertical de la souris, rotation autour de “right” (élévation)
- Molette de la souris, distance entre l’origine et la caméra



On considère que la position de la caméra est située sur une sphère de rayon R. Il est alors plus pratique de raisonner en coordonnées sphériques. Les mouvements des axes de la souris vont s’ajouter aux angle de rotation.
Utilisez la formule suivante pour déduire les coordonnées cartésiennes (X,Y,Z) à partir des coordonnées polaire (R, phi, theta), où “R” est le rayon, “phi” l’azimut et “theta” l’élévation.

Y = R * sin(Theta)
X = R * cos(Theta) * cos(Phi)
Z = R * cos(Theta) * sin(Phi)

Utilisez ensuite la fonction LookAt() avec comme “target” la position de votre objet.

Attention ! Veillez à Limiter les angles horizontaux à (-PI, +PI), et verticaux à (-PI/2, +PI/2).

Note: on peut également implémentez cette fonction sous la forme d’une rotation avec pivot. Les étapes sont : appliquez la matrice (–Translation), puis appliquez les rotations à l’origine, puis appliquez la matrice (Translation).